

Step -1 Created a Storage Account with hierarchical namespace enabled.

Home > vaishnavidata
Storage account

Search

Overview

- Activity log
- Tags
- Diagnose and solve problems
- Access Control (IAM)
- Data migration
- Events
- Storage browser
- Partner solutions
- Resource visualizer
- Data storage
 - Containers
 - File shares
 - Queues
 - Tables

Upload Open in Explorer Delete Move Refresh Open in mobile CLI / PS Feedback

Essentials JSON View

Resource group (move) LabsKraft2027

Location southindia

Subscription (move) LabsKraft

Subscription ID d008c7cc-9795-4292-bf79-74ae52cda63d

Disk state Available

Tags (edit) Add tags

Performance Standard

Replication Locally-redundant storage (LRS)

Account kind StorageV2 (general purpose v2)

Provisioning state Succeeded

Disk state Created

8/30/2025, 9:49:14 AM

Properties Monitoring Capabilities (5) Recommendations (0) Tutorials Tools + SDKs

Data Lake Storage

Hierarchical namespace Enabled

Default access tier Hot

Security

Require secure transfer for REST API operations Enabled

Step - 2 Created a container named healthcare and uploaded the files in the respective folders.

Home > vaishnavidata
Storage account

Containers

Search

+ Add container Upload Refresh Delete Change access level Restore containers Edit columns

Search containers by prefix Only show active containers

Showing all 2 items

☐	Name	Last modified	Anonymous access level	Lease state
☐	\$logs	8/30/2025, 9:49:57 AM	Private	Available ...
☐	healthcare	8/30/2025, 9:51:33 AM	Private	Available ...

Step -3 Create Access Connector and copy the resource id to add in storage credentials while creating external location for the unity catalog.

Home > Access Connector for Azure Databricks >

**vaishnavi-connector**
Access Connector for Azure Databricks

Search

RefreshDelete

Overview

Activity log

Access control (IAM)

Tags

Resource visualizer

Settings

Automation

Help

Essentials

Resource group (move)
[LabsKraft2027](#)

Location
South India

Subscription (move)
[LabsKraft](#)

Subscription ID
d008c7cc-9795-4292-bf79-74ae52cda63d

Tags (edit)
[Add tags](#)

State
Succeeded

Resource ID
/subscriptions/d008c7cc-9795-4292-bf79-74ae52cda63d/resourceGroups/...

[JSON View](#)

Step-4 Give IAM permission of Storage Blob Data Contributor to the managed identity->Access Connector->vaishnavi-connector.

Home > vaishnavidata | Access Control (IAM) >

Add role assignment

Role

Members

Conditions

Review + assign

Showing a filtered list of roles because your permissions include a condition. [Learn more](#)
[View my access](#)

Selected role

Storage Blob Data Contributor

Assign access to

☐ User, group, or service principal

☒ Managed identity

Members

[+ Select members](#)

Name

Object ID

No members selected

Description

Optional

Review + assign

Previous

Next

Select managed identities

Some results might be hidden due to your ABAC condition.

Subscription *
LabsKraft

Managed identity
Access Connector for Azure Databricks (22)

va

shivamconnector

/subscriptions/d008c7cc-9795-4292-bf79-74ae52cda63d/resourceGroups/LabsKra...

Selected members:

vaishnavi-connector

/subscriptions/d008c7cc-9795-4292-bf79-74ae52cda63d/resourceGroups/...

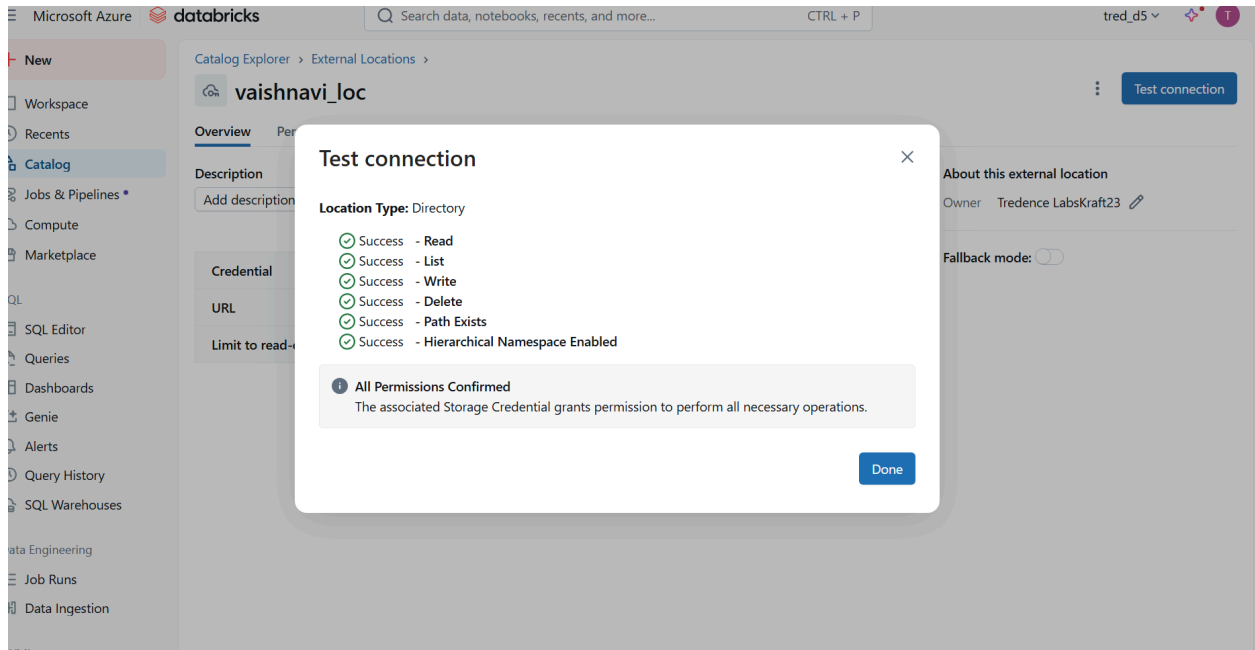
Remove

Select

Close

[Feedback](#)

Step - 5 Create external location and unity catalog -> healthcare_vaishnavi



Step -6 Creating bronze, silver and gold tables.

```
%sql
CREATE SCHEMA IF NOT EXISTS healthcare_vaishnavi.bronze;
CREATE SCHEMA IF NOT EXISTS healthcare_vaishnavi.silver;
CREATE SCHEMA IF NOT EXISTS healthcare_vaishnavi.gold;

import json
from pyspark.sql.types import *
from pyspark.sql.functions import *

# -----
# Step 0: Setup
# -----

catalog = "healthcare_vaishnavi"
schema = "bronze"

spark.sql(f"USE CATALOG {catalog}")
spark.sql(f"CREATE SCHEMA IF NOT EXISTS {schema}")
spark.sql(f"USE {schema}")

# Raw data paths
```

```

raw_labs_data =
"abfss://healthcare@vaishnavidata.dfs.core.windows.net/labs_data"
raw_patients_data =
"abfss://healthcare@vaishnavidata.dfs.core.windows.net/patients_data"
raw_visits_data =
"abfss://healthcare@vaishnavidata.dfs.core.windows.net/visits_data"
raw_vitals_data =
"abfss://healthcare@vaishnavidata.dfs.core.windows.net/vitals_data"
raw_patient_doctor =
"abfss://healthcare@vaishnavidata.dfs.core.windows.net/patient_doctor"

```

Bronze Table

1) Bronze Labs Data

```

bronze_labsdata = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "csv")
    .option("header", "true")
    .option("cloudFiles.schemaLocation",
"abfss://healthcare@vaishnavidata.dfs.core.windows.net/schema/bronze_labs
data")
    .option("cloudFiles.inferColumnTypes", "true")
    .option("cloudFiles.schemaHints", "date DATE")
    .load(raw_labs_data)
    .withColumn("ingest_file", col("_metadata.file_path"))
)

(bronze_labsdata.writeStream
    .format("delta")
    .option("checkpointLocation",
"abfss://healthcare@vaishnavidata.dfs.core.windows.net/chkpt/bronze_labsd
ata")
    .outputMode("append")
    .trigger(availableNow=True)
    .table(f"{catalog}.{schema}.bronze_labsdata")
)

```

```
)
```

2) Bronze Patients Data

```
bronze_patientsdata = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "csv")
    .option("header", "true")
    .option("cloudFiles.schemaLocation",
"abfss://healthcare@vaishnavidata.dfs.core.windows.net/schema/bronze_patientsdata_new")
    .option("cloudFiles.inferColumnTypes", "true")
    .load(raw_patients_data)
    .withColumn("ingest_file", col("_metadata.file_path"))
)
(bronze_patientsdata.writeStream
    .format("delta")
    .option("checkpointLocation",
"abfss://healthcare@vaishnavidata.dfs.core.windows.net/chkpt/bronze_patientsdata_new")
    .outputMode("append")
    .trigger(availableNow=True)
    .table(f"{catalog}.{schema}.bronze_patientsdata")
)
```

3) Bronze Visits Data

```
bronze_visitsdata = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "csv")
    .option("header", "true")
    .option("cloudFiles.schemaLocation",
"abfss://healthcare@vaishnavidata.dfs.core.windows.net/schema/bronze_visitsdata")
)
    .option("cloudFiles.inferColumnTypes", "true")
    .option("cloudFiles.schemaHints", "admission_date DATE, discharge_date DATE")
```

```

        .load(raw_visits_data)
        .withColumn("ingest_file", col("_metadata.file_path"))
    )

(bronze_visitsdata.writeStream
    .format("delta")
    .option("checkpointLocation",
"abfss://healthcare@vaishnavidata.dfs.core.windows.net/chkpt/bronze_visitsdata"
    )
    .outputMode("append")
    .trigger(availableNow=True)
    .table(f"{catalog}.{schema}.bronze_visitsdata")
)

```

4) Bronze Vitals Data

```

bronze_vitalsdata = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "csv")
    .option("header", "true")
    .option("cloudFiles.schemaLocation",
"abfss://healthcare@vaishnavidata.dfs.core.windows.net/schema/bronze_vitalsdata"
    )
    .option("cloudFiles.inferColumnTypes", "true")
    .option("cloudFiles.schemaHints", "timestamp TIMESTAMP")
    .load(raw_vitals_data)
    .withColumn("ingest_file", col("_metadata.file_path"))
)

(bronze_vitalsdata.writeStream
    .format("delta")
    .option("checkpointLocation",
"abfss://healthcare@vaishnavidata.dfs.core.windows.net/chkpt/bronze_vitalsdata"
    )
    .outputMode("append")
    .trigger(availableNow=True)
    .table(f"{catalog}.{schema}.bronze_vitalsdata")
)

```

5) Bronze Patient Doctor Data

```
bronze_patient_doctor = (spark.readStream
    .format("cloudFiles")
    .option("cloudFiles.format", "csv")
    .option("header", "true")
    .option("cloudFiles.schemaLocation",
"abfss://healthcare@vaishnavidata.dfs.core.windows.net/schema/bronze_patient_doctor")
    .option("cloudFiles.inferColumnTypes", "true")
    # .option("cloudFiles.schemaHints", "timestamp TIMESTAMP")
    .load(raw_patient_doctor)
    .withColumn("ingest_file", col("_metadata.file_path")))
)

(bronze_patient_doctor.writeStream
    .format("delta")
    .option("checkpointLocation",
"abfss://healthcare@vaishnavidata.dfs.core.windows.net/chkpt/bronze_patient_doctor")
    .outputMode("append")
    .trigger(availableNow=True)
    .table(f"{catalog}.{schema}.bronze_patient_doctor")
)
```

Silver Tables

1) Silver Patients Data

```
patients_df =
spark.table("healthcare_vaishnavi.bronze.bronze_patientsdata")
# Replace 'INVALID_*' entries with None
patients_df_cleaned = patients_df.select([
    when(col(c).rlike("INVALID_*"), "Unknown").otherwise(col(c)).alias(c)
    for c in patients_df.columns
])
```

```

# Cast age to int and filter out unrealistic values
patients_df_cleaned = patients_df_cleaned.withColumn("age",
col("age").cast("int"))
patients_df_cleaned = patients_df_cleaned.filter((col("age").isNotNull())
& (col("age") <= 120))

# Standardize gender values
patients_df_cleaned = patients_df_cleaned.withColumn("gender",
    when(col("gender").isin("M", "F", "Other"),
col("gender")).otherwise("Other")
)

# Fill remaining nulls
patients_df_cleaned = patients_df_cleaned.fillna({
    "age": 50,
    "gender": "Other",
    "diagnosis": "Undiagnosed",
    "prescription": "None"
})

# Drop rows with missing patient_id or name
patients_df_cleaned = patients_df_cleaned.dropna(subset=["patient_id",
"name"])

# Remove duplicates
patients_df_cleaned = patients_df_cleaned.dropDuplicates()
patients_df_cleaned = patients_df_cleaned.filter(col("prescription") !=
"Unknown")

# Ensure all columns match the target schema
patients_df_cleaned = patients_df_cleaned.select(
    col("patient_id").cast("string"),
    col("name").cast("string"),
    col("age").cast("int"),
    col("gender").cast("string"),
    col("diagnosis").cast("string"),
    col("prescription").cast("string")
)

```



```
patients_df_cleaned.write \
    .format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable("healthcare_vaishnavi.silver.silver_patientsdata")
```

2) Silver Labs Data

```
labs_df = spark.table("healthcare_vaishnavi.bronze.bronze_labsdata")
```

```
labs_cleaned = labs_df.select([
    when(col(c).rlike("^INVALID_.*"),
"Unknown").otherwise(col(c)).alias(c)
    for c in labs_df.columns
])
```

```
labs_cleaned = labs_cleaned.dropna(subset=["patient_id",
"test_name", "date"])
```

```
labs_cleaned = labs_cleaned.fillna({"unit": "Unspecified"})
```

```
labs_cleaned = labs_cleaned.withColumn("result",
col("result").cast("float"))
```

```
labs_cleaned = labs_cleaned.filter((col("result").isNotNull()) &
(col("result") >= 0) & (col("result") <= 1000))
```

```
labs_cleaned = labs_cleaned.dropDuplicates()
```

```
labs_cleaned = labs_cleaned.select(
    col("patient_id").cast("string"),
    col("test_name").cast("string"),
    col("result").cast("float"),
    col("unit").cast("string"),
    col("date").cast("date")
)
```

```

labs_cleaned.write \
    .mode("overwrite") \
    .format("delta") \
    .saveAsTable("healthcare_vaishnavi.silver.silver_labsdata")

```

3) Silver visits data

```

visits_df = spark.table("healthcare_vaishnavi.bronze.bronze_visitsdata")

visits_cleaned = visits_df.select([
    when(col(c).rlike("^INVALID_.*"),
"Unknown").otherwise(col(c)).alias(c)
    for c in visits_df.columns
])

visits_cleaned = visits_cleaned.dropna(subset=["patient_id",
"admission_date"])

visits_cleaned = visits_cleaned.fillna({"reason": "Unspecified"})
visits_cleaned = visits_cleaned.filter(col("reason") != "Unknown")

visits_cleaned = visits_cleaned.dropDuplicates()

visits_cleaned = visits_cleaned.select(
    col("patient_id").cast("string"),
    col("admission_date").cast("date"),
    col("discharge_date").cast("date"),
    col("reason").cast("string")
)

visits_cleaned = visits_cleaned.withColumn(
    "is_active",
    when(col("discharge_date").isNull(), True).otherwise(False)
)

visits_cleaned.write \

```

```

.mode("overwrite") \
.format("delta") \
.saveAsTable("healthcare_vaishnavi.silver.silver_visitsdata")

```

4) Silver vitals data

```

df = spark.table("healthcare_vaishnavi.bronze.bronze_vitalsdata ")

df_cleaned = df.select([
    when(col(c).rlike("^INVALID_.*"),
"Unknown").otherwise(col(c)).alias(c)
    for c in df.columns
])

df_cleaned = df_cleaned.dropna(subset=["patient_id", "timestamp"])

df_cleaned = df_cleaned.fillna({
    "device_id": "Unspecified",
    "vital_type": "Unspecified"
})

# Cast vital_value to float and filter out unrealistic values
df_cleaned = df_cleaned.withColumn("vital_value",
col("vital_value").cast("float"))
df_cleaned = df_cleaned.filter((col("vital_value").isNotNull()) &
(col("vital_value") >= 0) & (col("vital_value") <= 1000))

df_cleaned = df_cleaned.filter(col("vital_type") != "Unspecified")
df_cleaned = df_cleaned.filter(col("device_id") != "Unspecified")

df_cleaned = df_cleaned.dropDuplicates()

```

```

# Ensure schema consistency
df_cleaned = df_cleaned.select(
    col("patient_id").cast("string"),
    col("device_id").cast("string"),
    col("timestamp").cast("timestamp"),
    col("vital_type").cast("string"),
    col("vital_value").cast("float")
)

df_cleaned.write \
    .format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable("healthcare_vaishnavi.silver.silver_vitalsdata")

```

5) Silver patient doctor map data

```

doctor_map_df =
spark.table("healthcare_vaishnavi.bronze.bronze_patient_doctor")

doctor_map_cleaned = doctor_map_df.select([
    when(col(c).rlike("INVALID_*"), "Unknown").otherwise(col(c)).alias(c)
    for c in doctor_map_df.columns
])

doctor_map_cleaned =
doctor_map_cleaned.dropna(subset=["patient_id", "doctor_id"])

doctor_map_cleaned = doctor_map_cleaned.fillna({"care_team":
"Unassigned"})

doctor_map_cleaned = doctor_map_cleaned.filter(col("care_team") !=
"Unknown")

doctor_map_cleaned = doctor_map_cleaned.filter(col("doctor_id") !=
"Unknown")

doctor_map_cleaned = doctor_map_cleaned.dropDuplicates()

```

```

doctor_map_cleaned = doctor_map_cleaned.select(
    col("patient_id").cast("string"),
    col("doctor_id").cast("string"),
    col("care_team").cast("string")
)

doctor_map_cleaned.write \
    .format("delta") \
    .mode("overwrite") \
    .saveAsTable("healthcare_vaishnavi.silver.silver_patient_doctor")

```

Gold Tables

1) Patient Profile Data in Raw and clean(for BI)

```

# Base join
patient_profile_base = (
    spark.table("healthcare_vaishnavi.silver.silver_patientsdata")

    .join(spark.table("healthcare_vaishnavi.silver.silver_patient_doctor"),
    "patient_id", "left")
)

# Cleaned version for dashboards
patient_profile_clean = (
    patient_profile_base
    .fillna({"doctor_id": "Unassigned", "care_team": "General"})
)

# Save as Gold table for BI
patient_profile_clean.write.mode("overwrite").saveAsTable(
    "healthcare_vaishnavi.gold.gold_patient_profile_clean"
)

# Raw version (keep nulls for ML/analytics)
patient_profile_raw = patient_profile_base

```

```
# Save as Gold table for ML
patient_profile_raw.write.mode("overwrite").saveAsTable(
    "healthcare_vaishnavi.gold.gold_patient_profile_raw"
)
```

2) Latest Lab Results

```
from pyspark.sql import Window

# Window to pick latest lab test per patient
w = Window.partitionBy("patient_id",
    "test_name").orderBy(col("date").desc())

lab_trends_df = (
    spark.table("healthcare_vaishnavi.silver.silver_labsdata")
    .withColumn("rn", row_number().over(w))
    .filter(col("rn") == 1)
    .drop("rn")
)

# Save as Gold table
lab_trends_df.write.format("delta").mode("overwrite").saveAsTable("health
care_vaishnavi.gold.latest_lab_results")
```

3) Visits Summary

```
visit_summary_df = (
    spark.table("healthcare_vaishnavi.silver.silver_visitsdata")
    .groupBy("patient_id")
    .agg(
        count("*").alias("total_visits"),
        sum(when(col("is_active"),
1).otherwise(0)).alias("active_visits"),
        sum(when(~col("is_active"),
1).otherwise(0)).alias("completed_visits"),
        avg(datediff("discharge_date",
"admission_date")).alias("avg_stay_length"),
```

```

        max("discharge_date").alias("last_discharge")
    )
)

visit_summary_df.write.format("delta").mode("overwrite").saveAsTable("healthcare_vaishnavi.gold.visit_summary")

```

4) Vitals summary

```

vitals_daily_df = (
    spark.table("healthcare_vaishnavi.silver.silver_vitalsdata")
    .withColumn("date", to_date("timestamp"))
    .groupBy("patient_id", "vital_type", "date")
    .agg(
        avg("vital_value").alias("avg_value"),
        max("vital_value").alias("max_value"),
        min("vital_value").alias("min_value")
    )
)

vitals_daily_df.write.format("delta").mode("overwrite").saveAsTable("healthcare_vaishnavi.gold.vitals_daily")

```

5) Risk Features

```

from pyspark.sql import functions as F

# Rename 'date' in vitals
vitals_df = spark.table("healthcare_vaishnavi.gold.vitals_daily") \
    .withColumnRenamed("date", "vitals_date")

# Rename 'date' in labs
labs_df = spark.table("healthcare_vaishnavi.gold.latest_lab_results") \
    .withColumnRenamed("date", "lab_date")

risk_features_df = (
    spark.table("healthcare_vaishnavi.gold.gold_patient_profile_raw")
    .join(vitals_df, "patient_id", "left")

```

```

        .join(labs_df, "patient_id", "left")
        .join(spark.table("healthcare_vaishnavi.gold.visit_summary"),
"patient_id", "left")
    )

risk_features_df.write.format("delta").mode("overwrite").saveAsTable(
    "healthcare_vaishnavi.gold.risk_features"
)

```

6) Medication Summary

```

from pyspark.sql import functions as F

# Read silver patients table
patients_df =
spark.table("healthcare_vaishnavi.silver.silver_patientsdata")

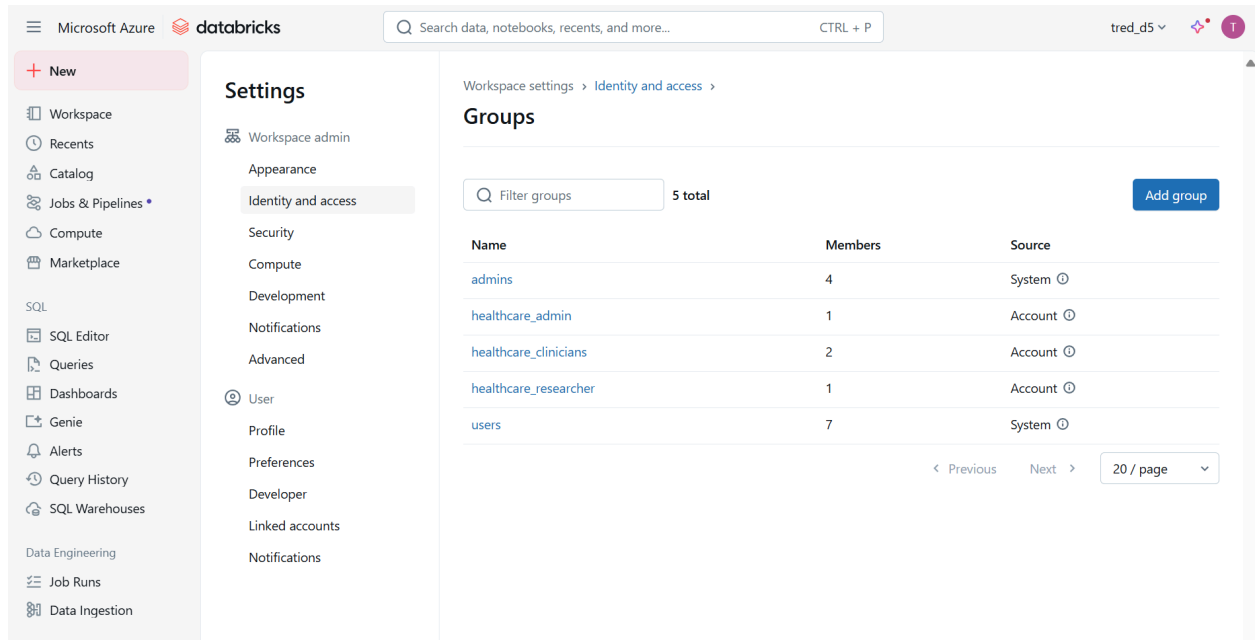
# Split prescriptions into array (assuming comma-separated)
med_summary_df = (
    patients_df
        .withColumn("med_list", F.split(F.col("prescription"), ",\\s*"))
        .withColumn("diagnosis_list", F.split(F.col("diagnosis"), ",\\s*"))
        .withColumn("total_medications", F.size(F.col("med_list")))
        .withColumn("latest_prescription", F.element_at(F.col("med_list"),
-1)) # last item in list
        .select("patient_id", "diagnosis_list", "total_medications",
"latest_prescription")
    )

# Save as Gold table
med_summary_df.write.format("delta").mode("overwrite").saveAsTable("healthcare_vaishnavi.gold.medication_summary")

```

Permissions

Created groups in databricks workspace.



Grant all permissions to admin

```
%sql
GRANT ALL PRIVILEGES ON SCHEMA healthcare_vaishnavi.bronze TO
`healthcare_admin`;
GRANT ALL PRIVILEGES ON SCHEMA healthcare_vaishnavi.silver TO
`healthcare_admin`;
GRANT ALL PRIVILEGES ON SCHEMA healthcare_vaishnavi.gold TO
`healthcare_admin`;
```

If we had login mails then we could have allowed particular doctor to view only his patient.

```
%sql
CREATE OR REPLACE TABLE healthcare_vaishnavi.gold.doctor_user_mapping (
  doctor_id STRING,          -- used in patient records
  user_principal_name STRING -- login/email that current_user() returns
);

-- Insert 5 doctors
INSERT INTO healthcare_vaishnavi.gold.doctor_user_mapping VALUES
  ('D001', 'vaishnavi.m@hospital.org'),
  ('D002', 'amit.k@hospital.org'),
  ('D003', 'sneha.r@hospital.org'),
```

```
('D004', 'rahul.s@hospital.org'),  
('D005', 'priya.t@hospital.org');
```

```
%sql  
CREATE OR REPLACE VIEW healthcare_vaishnavi.gold.secure_patient_profile AS  
SELECT p.*  
FROM healthcare_vaishnavi.gold.gold_patient_profile_clean p  
JOIN healthcare_vaishnavi.gold.doctor_user_mapping m  
  ON p.doctor_id = m.doctor_id  
WHERE m.user_principal_name = current_user();
```

Grant only limited permission to clinicians

```
%sql  
GRANT SELECT ON VIEW healthcare_vaishnavi.gold.secure_patient_profile TO  
`healthcare_clinicians`;
```

Hiding PII from Researchers

```
%sql  
CREATE OR REPLACE VIEW healthcare_vaishnavi.gold.research_patient_profile AS  
SELECT patient_id,  
       SHA2(name, 256) AS anonymized_name,  
       age,  
       gender,  
       diagnosis,  
       prescription  
FROM healthcare_vaishnavi.gold.gold_patient_profile_clean;
```

```
%sql  
GRANT SELECT ON VIEW healthcare_vaishnavi.gold.research_patient_profile TO  
`healthcare_researcher`;
```

Only admin can view all the information not everyone.

```
%sql
CREATE OR REPLACE FUNCTION healthcare_vaishnavi.gold.mask_name(name STRING)
RETURNS STRING
RETURN CASE
    WHEN is_member('healthcare_admin') THEN name
    ELSE concat('Patient_', sha2(name, 256))
END;
```

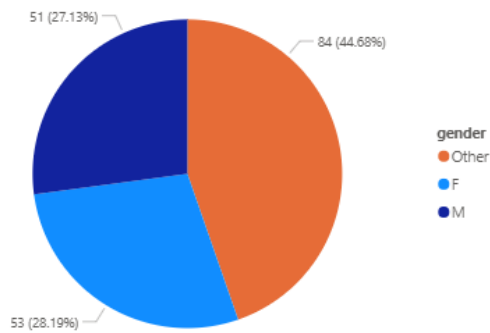
```
%sql
SELECT patient_id, healthcare_vaishnavi.gold.mask_name(name), age, gender
FROM healthcare_vaishnavi.silver.silver_patientsdata;
```

Retention Policy of 10 years

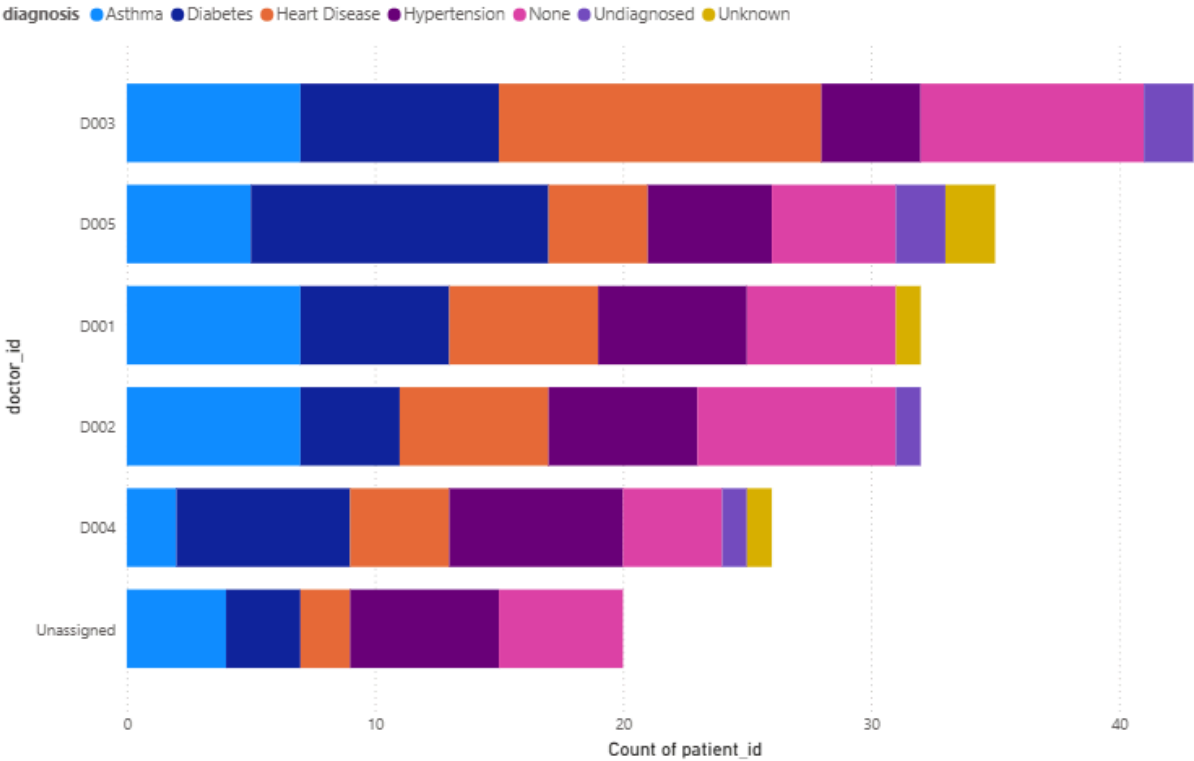
```
%sql
ALTER TABLE healthcare_vaishnavi.silver.silver_patientsdata
SET TBLPROPERTIES (
    'delta.logRetentionDuration' = '3650 days',
    'delta.deletedFileRetentionDuration' = '3650 days'
);
```

PowerBi visualizations

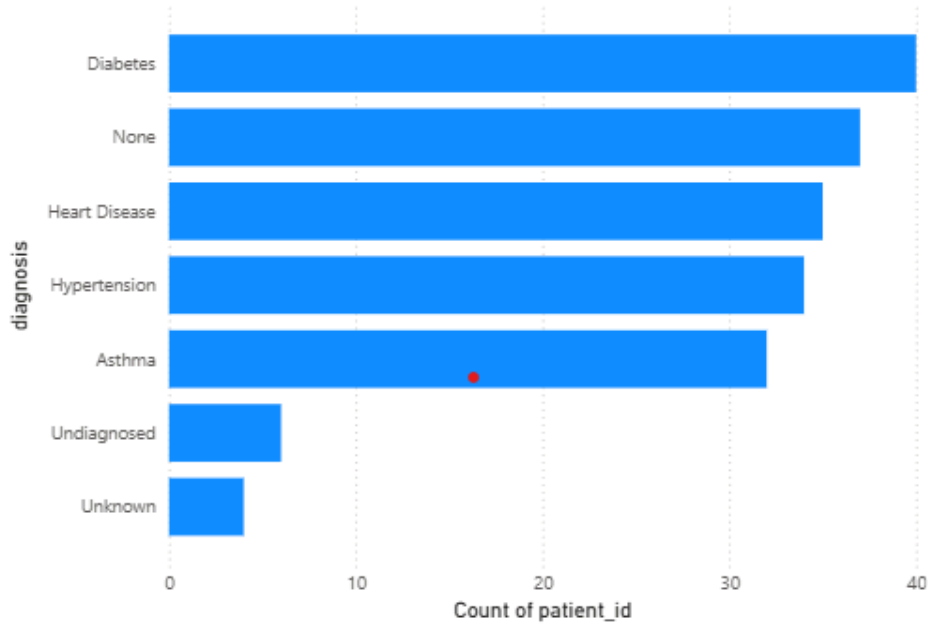
Count of patient_id by gender



Count of patient_id by doctor_id and diagnosis

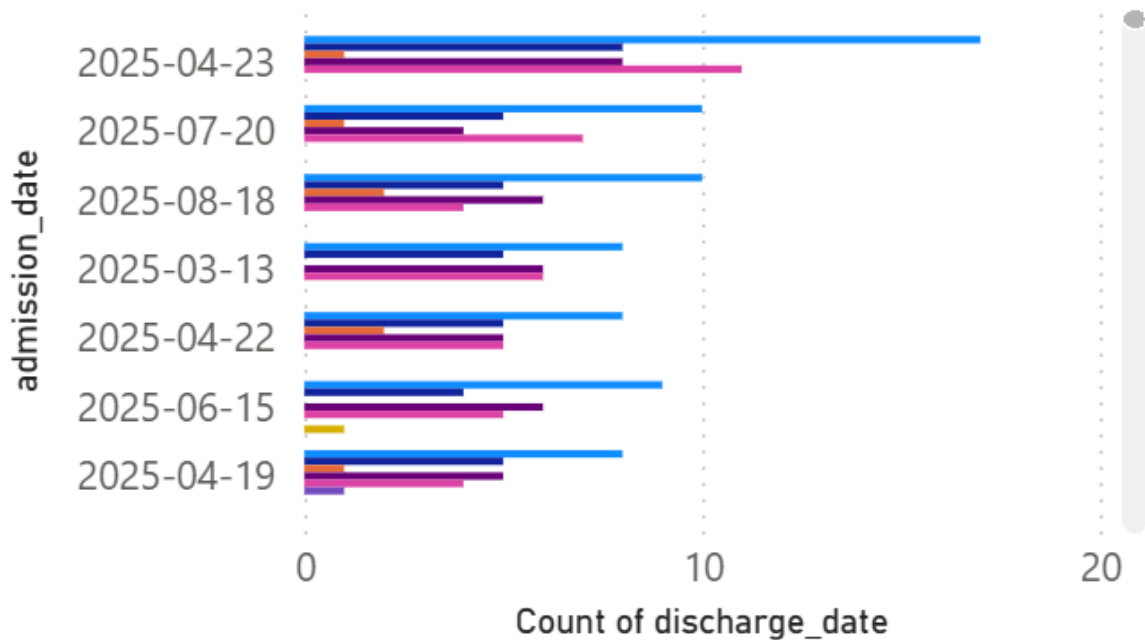


Count of patient_id by diagnosis

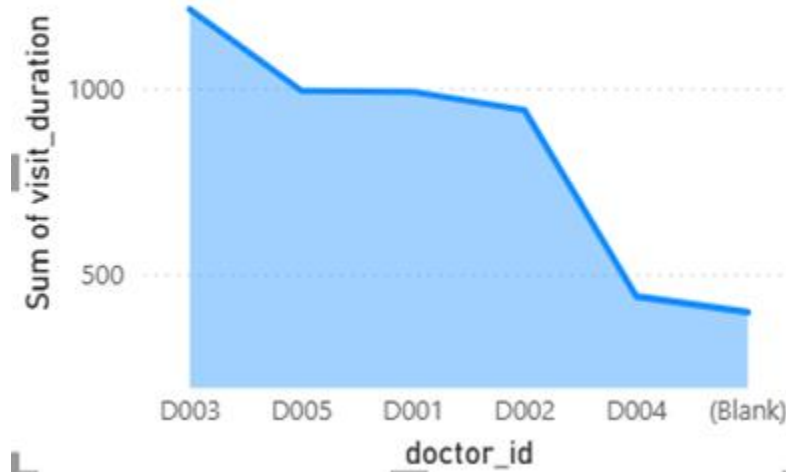


Count of discharge_date by admission_date and diagnosis

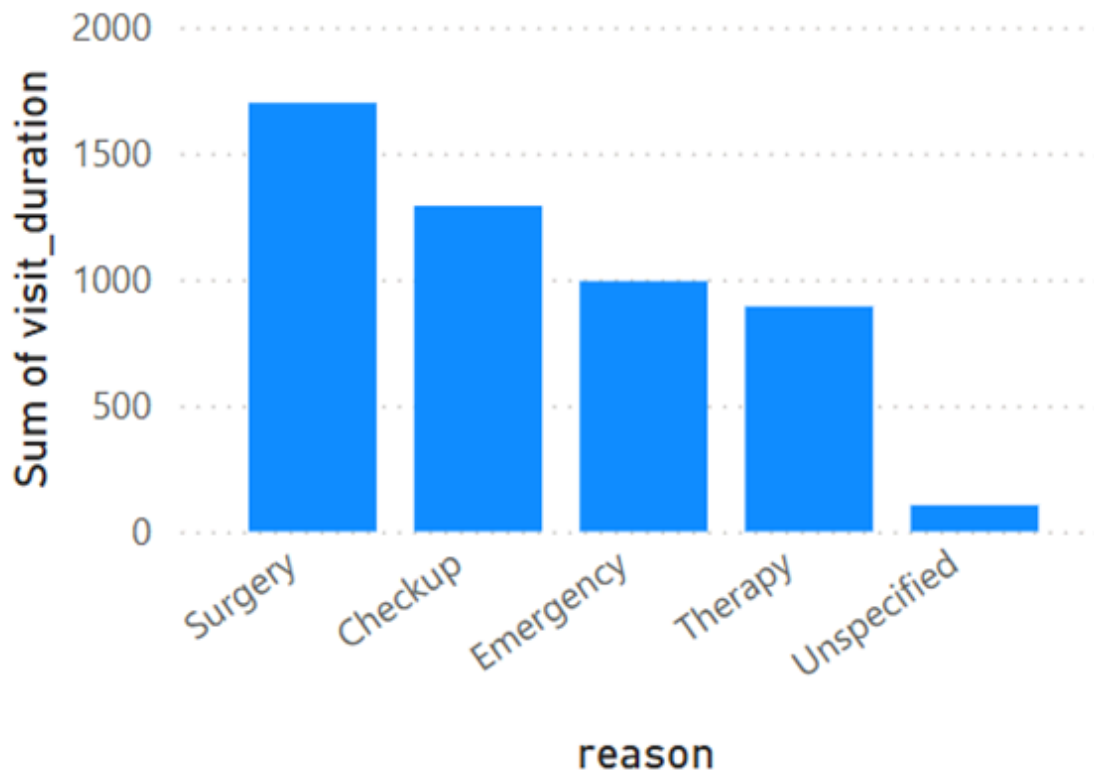
diagnosis ● Asthma ● Diabetes ● Heart Disease



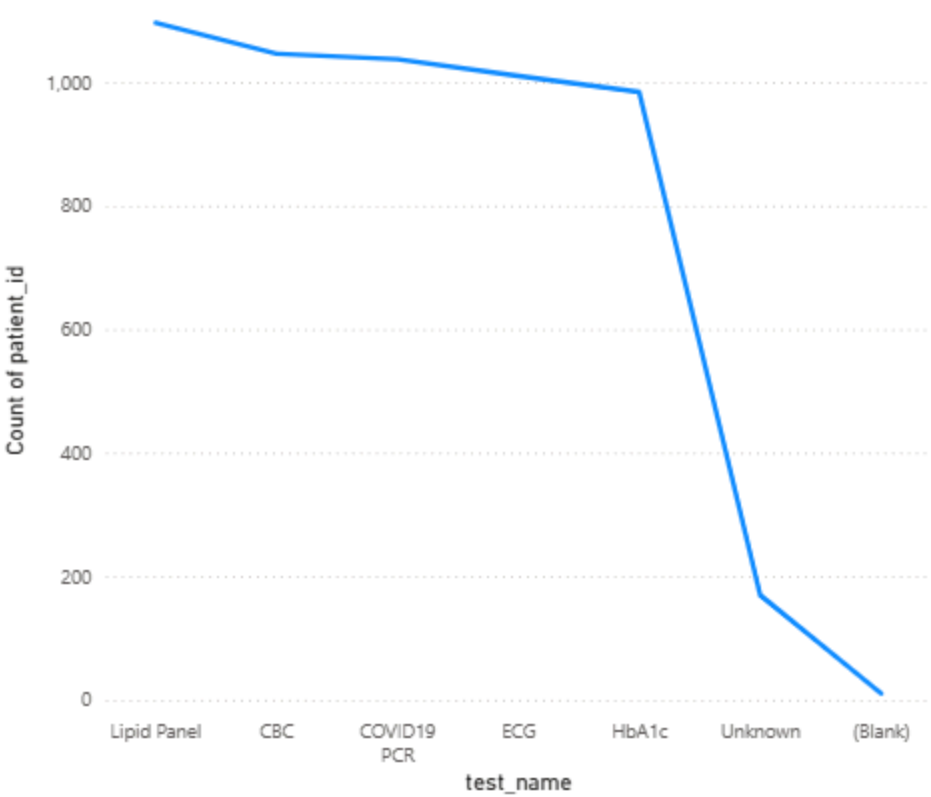
Sum of visit_duration by doctor_id



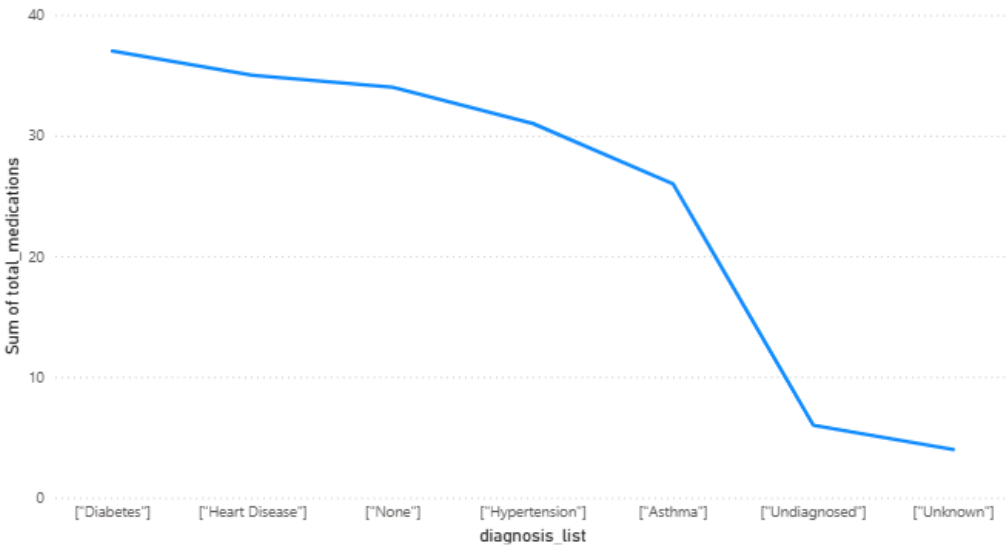
Sum of visit_duration by reason



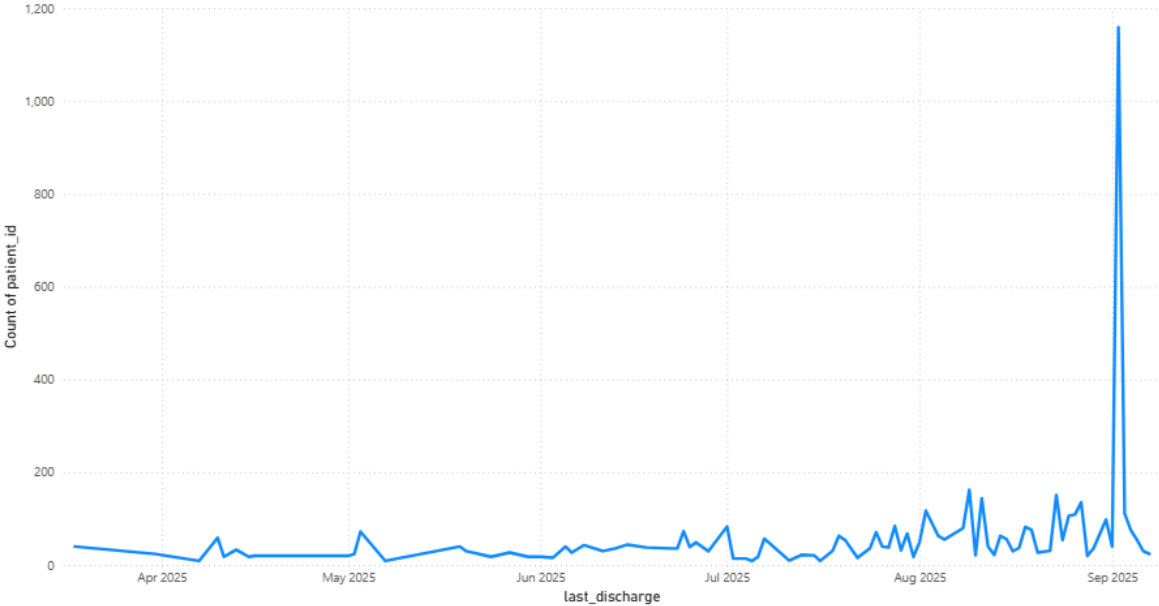
Count of patient_id by test_name



Sum of total_medications by diagnosis_list



Count of patient_id by last_discharge



Streaming Vitals Data

Event Hub creation

Home > **vaishnavieventhub** Event Hubs Namespace

How do I troubleshoot issues with this Event Hubs Namespace? List current consumers of this Event Hubs Namespace. +1

Search

Overview

- Activity log
- Access control (IAM)
- Tags
- Diagnose and solve problems
- Data Explorer
- Resource visualizer
- Events
- Settings
- Entities
- Monitoring
- Automation
- Help

+ Event Hub Delete Refresh Give feedback

You can start generating test data or inspect data that has already been sent with the new Azure Event Hubs Data Explorer. Click on this message to try the feature!

Essentials

Resource group (move) [LabsKraft2027](#)

Status **Succeeded**

Location **South India**

Subscription (move) [LabsKraft](#)

Subscription ID **d008c7cc-9795-4292-bf79-74ae52cda63d**

Host name **vaishnavieventhub.servicebus.windows.net**

Created **Sunday, August 31, 2025 at 14:23:04 GMT+5:30**

Updated **Sunday, August 31, 2025 at 14:23:24 GMT+5:30**

Zone Redundancy **Not Enabled**

Pricing tier **Basic**

Throughput Units **1 unit**

Auto-inflate throughput units **Not Supported**

Local Authentication **Enabled**

Tags (edit) [Add tags](#)

NAMESPACE CONTENTS **0 EVENT HUBS** KAFKA SURFACE **NOT SUPPORTED**

Add or remove favorites by pressing Ctrl+L+Shift+F

Home > **vaishnavieventhub** > **vmhub (vaishnavieventhub/vmhub)**

vmhub (vaishnavieventhub/vmhub) | Shared access policies Event Hubs Instance

Search

Overview

- Access control (IAM)
- Diagnose and solve problems
- Data Explorer
- Resource visualizer
- Settings
- Shared access policies**
- Configuration
- Properties
- Locks
- Entities
- Features
- Automation
- Help

+ Add

Search to filter items...

Policy

[Manage](#)

SAS Policy: Manage

Event Hub

Regenerate primary key Regenerate secondary key Delete

Event Hubs access control with Shared Access Signatures [Learn more](#)

Claims

- ☒ Manage
- ☒ Send
- ☒ Listen

Primary key

Secondary key

Primary connection string

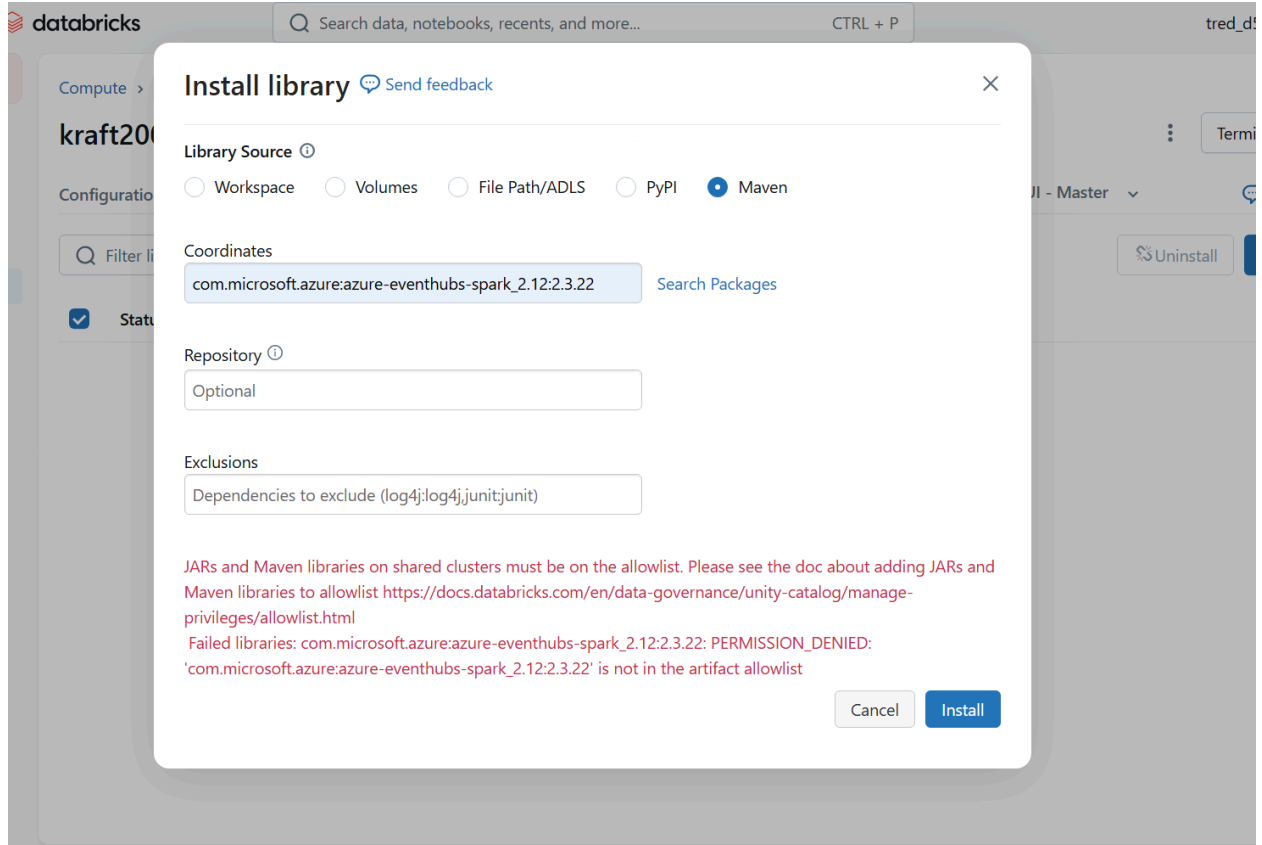
Secondary connection string

SAS policy ARM ID

/subscriptions/d008c7cc-9795-4292-bf79-74ae52cda63d/resourceGroups/labsKraft

Save Discard changes Give feedback

In the connection -> add shared access policies ->copy the connection string.



```
# Databricks notebook (PySpark)
from pyspark.sql.types import StructType, StructField, StringType, DoubleType,
TimestampType
from pyspark.sql.functions import from_json, col

# 1. declare schema for vitals JSON payload
vitals_schema = StructType([
    StructField("patient_id", StringType(), True),
    StructField("device_id", StringType(), True),
    StructField("vital_type", StringType(), True),      # e.g., heart_rate
    StructField("vital_value", DoubleType(), True),
    StructField("timestamp", TimestampType(), True),   # device timestamp
])

# 2. Event Hubs configuration
eventhub_conf = {

    "Endpoint=sb://vaishnavieventhub.servicebus.windows.net/;SharedAccessKeyName=Ma
```

```

nage;SharedAccessKey=H/7hwApDmeZLfHHkYp4QxhQcrsnzmGnxt+AEhJEugow=;EntityPath=vm
hub"
}

```

```

# 3. Read stream

```

```

eh_raw = (spark.readStream
    .format("eventhubs")
    .option("eventhubs.connectionString",connection_string)
    .load()
)

```

```

# 4. Extract body as string, parse JSON

```

```

raw_vitals = (eh_raw
    .selectExpr("cast(body as string) as body_str")
    .select(from_json(col("body_str"), vitals_schema).alias("data"))
    .select("data.*")
    .withColumn("ingest_received_ts", col("event_time")) # optional
extra ts
)

```

```

# 5. Write to Bronze Delta table with checkpointing

```

```

bronze_table = "healthcare_vaishnavi.bronze.bronze_vitals_eventhub"
checkpoint_path =
"abfss://healthcare@vaishnavidata.dfs.core.windows.net/chkpt/bronze_vitals_eventhub"

```

```

(bronze_raw := raw_vitals).writeStream \
    .format("delta") \
    .option("checkpointLocation", checkpoint_path) \
    .outputMode("append") \
    .trigger(processingTime="10 seconds") \    # micro-batches every 10s; tune
to get <30s alerts
    .table(bronze_table)

```

❶ > [UNSUPPORTED_STREAMING_SOURCE_PERMISSION_ENFORCED] Data source eventhubs is not supported as a streaming source on a shared cluster. SQLSTATE: 0A000

